

enough extra messages can emulate public-key-based mechanisms; therefore, these proofs are omitted here. Meanwhile, the formalization of macaroons in authorization logic has been more clarifying, and is presented in the appendix.

HMAC-based macaroons do suffer in one important way compared to public-key-based credentials: only the target service that originally mints a macaroon can check its validity. Specifically, the verification of an HMAC-based macaroon requires knowledge of its root key—and since this key confers the ability to arbitrarily modify the macaroon and its caveats, it cannot be widely shared. On the other hand, third-party verification is possible when using asymmetric signing keys and public verification keys, as in SPKI/SDSI, or the public-key-based macaroons described later in Section V-H. However, the HMAC-based macaroons are orders-of-magnitude more efficient (as shown in Section VII), which, in many cases, is a worthwhile trade-off.

Macaroons' flexibility does not obviate the need to analyze the security of any concrete authorization protocols using them—just as with any other type of authorization credential. In particular, such protocols must account for how target services learn, during verification, the intermediate signatures of macaroons, as well as the root keys of all caveat discharge macaroons, and consider how—in an adversarial model—other parties may also learn those keys, e.g., due to flaws in target services. Similarly, the concrete means for establishing macaroon key identifiers must be analyzed, especially if shared symmetric keys are used to add third-party caveats. Fortunately, many frameworks exist for the analysis of authorization protocols that use public-key, shared-key, and MAC-based constructions [45, 10, 4]. Also, such analysis can be greatly simplified if fresh, pairwise symmetric keys establish trust between principals and if macaroons use caveats with fresh, high-entropy root keys that are unique and distinct from root keys in other macaroons. For this purpose, protocols may choose to derive root keys using the entropy of intermediate macaroon signatures.

V. IMPLEMENTATION CONCERNS

This section considers how distributed systems may implement macaroon-based authorization. The choice of topics is based on experiences building prototype macaroon mechanisms for Cloud clients, servers, and mobile devices, and using these to implement both new authorization protocols and to improve existing ones, like those based on OAuth2 [17].

Empirically, in distributed systems, macaroons are simple to implement, easy to integrate with existing code, and incur little overhead. In particular, it simplifies the task that much of macaroons' flexibility stems from the semantics of caveats being left up to application services. However, macaroon-based systems still need to support a common library of well-known caveats—such as for specifying expiration time, as well as predicates about client context such as IP addresses, ChannelID, etc.—leaving service- or application-specific caveat predicates to be used in specific protocols.

A. Encoding Key Identifiers

The key identifiers for both macaroons and caveats may be constructed using service-specific means, as discussed in

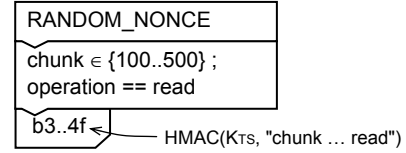


Fig. 9. Multiple caveats under one signature, as an optimization.

Section IV. For example, root keys, lists of predicates, or other secrets may be stored in a database at the service and identified by an index key (such as a random nonce, like in Section III). Such identifiers have the nice property of being completely opaque and allowing revocation to be simply done by deleting a database row; however, the downside is that the target must maintain a database, either centralized or replicated across the servers operating the service.

Alternatively, one may minimize state kept at the principal's server nodes by encrypting the keys and other secrets to be identified. For this, shared, symmetric encryption keys may be used, known to and trusted by each principal pair; alternatively, public-key encryption may be used, which can allow the same key to be used by several principals to create caveats for a single service that holds the private key.

B. Minting Caveat Root Keys

The root keys for caveats and caveat discharge macaroons are shared across principals; in particular, the root keys of all verified macaroons become known to the target service for a request. Thus, such root keys should not be reused—e.g., by embedding the same caveat identifier in multiple macaroons—unless this is strictly necessary (e.g., for performance reasons), and it is done in a known-to-be secure fashion (e.g., verified by a separate protocol analysis). In particular, while such reuse may be safe as long as all macaroons are for the same target service, it's most likely not safe when multiple target services are involved.

Therefore, to add a third-party caveat to a macaroon, an unpredictable caveat root key must be chosen—otherwise malicious parties might guess the root key and forge a discharge macaroon—even though some principals (e.g., Web browser frames, or embedded devices), may not have a good source of entropy for creating truly random keys.

To address this concern, caveat root keys may be derived from the signature of the embedding macaroon. Concretely, for adding a caveat to a macaroon with signature sig , its bearer may generate a strong root key using $HMAC(sig, n)$ —where n is an arbitrary nonce—because HMAC is a pseudo-random function [43]. Such a key will be unique, due to the second pre-image resistance of HMAC, and will therefore never be repeated across different caveats.

C. Optimizations and Privacy

Macaroons permit many optimizations, in both their primitives and applications. For example, a reduced number of operations can be used for chained-MACs, if caveats are combined as in Figure 9.

Both as an optimization, and also to protect privacy, applications and services may choose to omit certain information from caveat discharge macaroons. In particular, for nested